Django API Rest

PARTE 1: Creación del proyecto

1. Descargar e Instalar Python (https://www.python.org/downloads/)
2. Crear un nuevo folder llamado django-project
3. Abril el folder en Visual Studio Code (VSC)
4. Abrir terminar de comando en VSC
5. Crear un nuevo entorno virtual dentro del folder creado
   py -m venv venv

6. Activar el entorno virtual

   .\venv\Scripts\activate

7. Revisar que paquetes están instalados por defecto
   pip list

8. Instalar Django
   pip install django

9. Crear proyecto
   django-admin startproject tienda_online . (Nota: el punto al final es para no crear sub carpeta del proyecto)

10. Instalar django rest framework (https://www.django-rest-framework.org/#installation)
    pip install djangorestframework
    pip install markdown
    pip install django-filter

11. Agregar 'rest_framework' al INSTALLED_APPS en el archivo setting.py del proyecto
    En este paso para mantener más ordenadas las apps instaladas en el proyecto, se van a crear tres diferentes variables.
    BASE_APPS (Sera para todas las aplicaciones que están por defecto)
    THIRD_APPS (Sera para las aplicaciones de terceros)
    OWN_APPS (Sera para las aplicaciones que crearemos más adelante)

12. Crear la aplicación de productos
    a. Crear la app: py manage.py startapp products
    b. Agregar 'products' al OWN_APPS en el archivo setting.py del proyecto

```python
BASE_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]

THIRD_APPS = [
    'rest_framework',
]

OWN_APPS = [
    'products',
]

INSTALLED_APPS = BASE_APPS + THIRD_APPS + OWN_APPS
```

PARTE 2: Conectar MySQL al proyecto

1. Instalar la librería mysqlclient
   pip install mysqlclient

2. Crear la base de datos en Workbench
   CREATE DATABASE django_api;

3. Procedemos a modificar el archivo settings.py. Indicamos que haremos uso del gestor MySQL sobre sqlite.

```
DATABASES = {
   'default': {
      'ENGINE': 'django.db.backends.mysql',
      'NAME': 'hackathon',
      'USER': 'root',
      'PASSWORD': '',
      'HOST': 'localhost',
      'PORT': '',
   }
}
```

4. Ejecutar las migraciones
   py manage migrate

   Si te aparece el siguiente error "mariadb 10.5 or later is required (found 10.4.28)"
   Para solucionarlo seguir los pasos de la siguiente URL
   https://stackoverflow.com/questions/74233986/how-to-solve-django-db-utils-notsupportederror-in-django

5. Crear el super usuario
   a. *python manage.py createsuperuser*
   b. *Ingresar nombre, correo y contraseña*
   c.
6. Correr el servidor
   python manage.py runserver

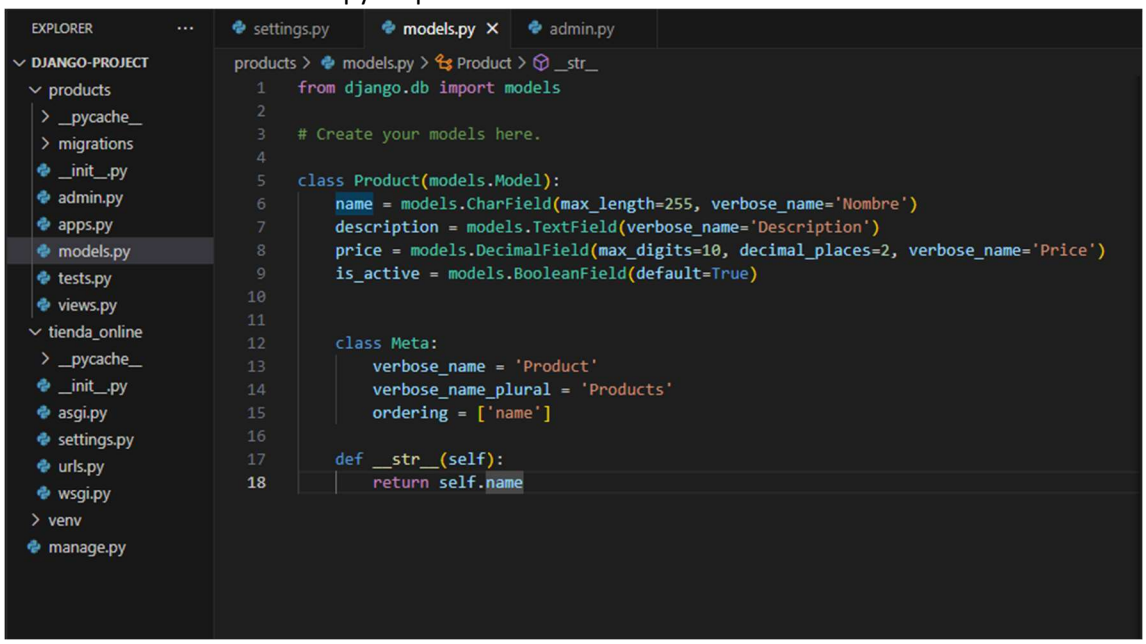
Para poder ver el administrado ingresar a la siguiente URL

http://127.0.0.1:8000/admin

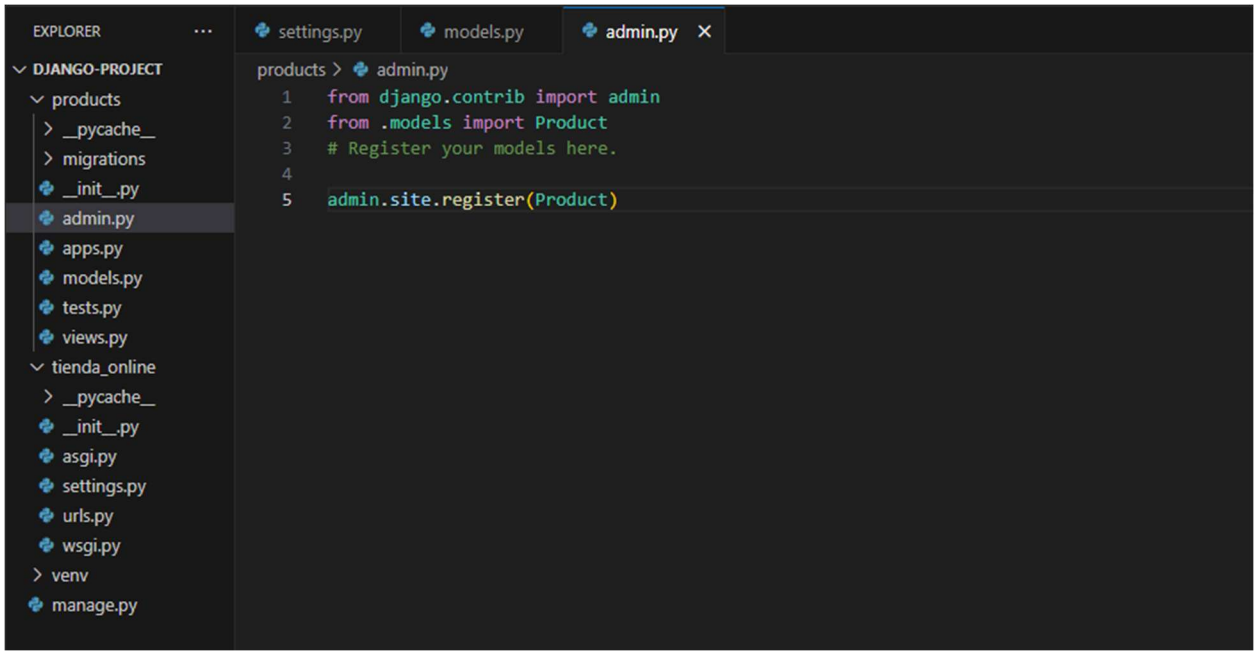
Ingresaran con el super usuario creado en el paso anterior

PARTE 3: Crear el modelo de productos

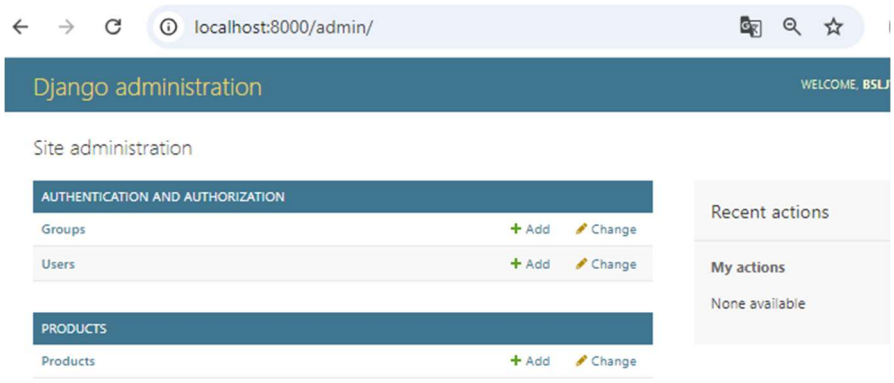
1. Modificar el archivo models.py de products



2. Agregar el modelo en el archivo admin.py para que el modelo producto aparezca en el administrador



3. Crear las migraciones
   python manage.py makemigrations
   python manage.py migrate

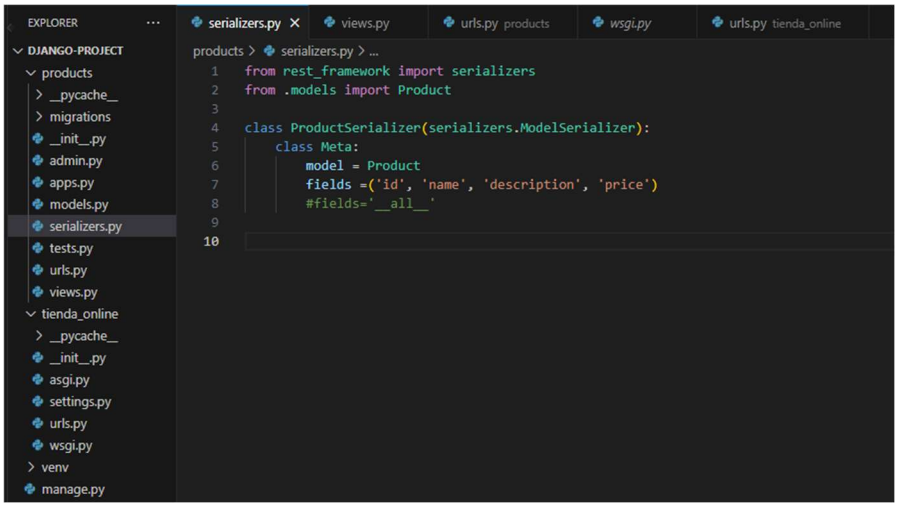


PARTE 4: Agregar Rest Framework a Django

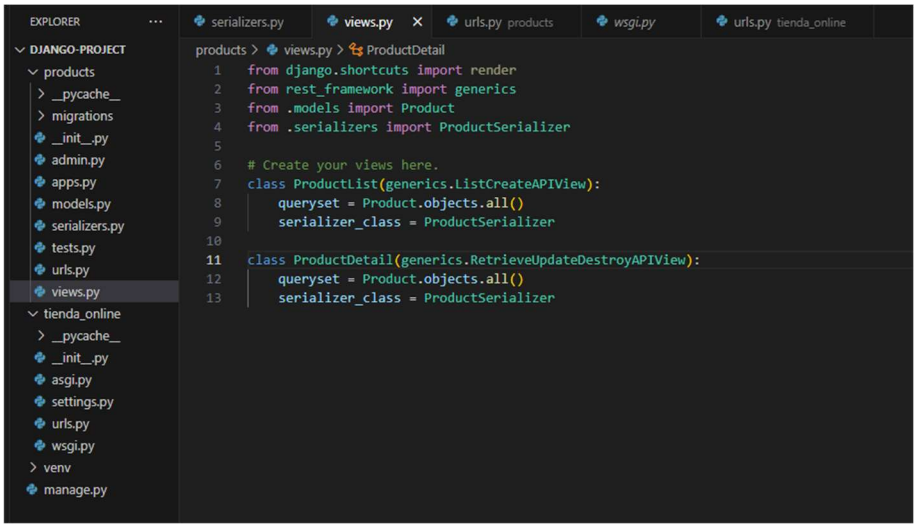
1. Crear el serializador

   El serializador se encarga de transformar la información del modelo a un formato estándar, comumente JSON.

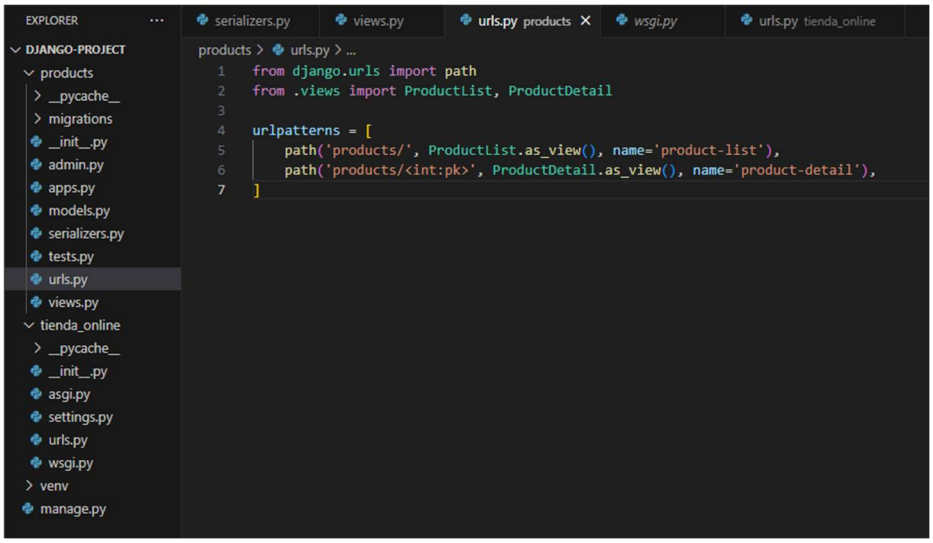
   a. Crear nuevo archivo serializers.py en el folder de products



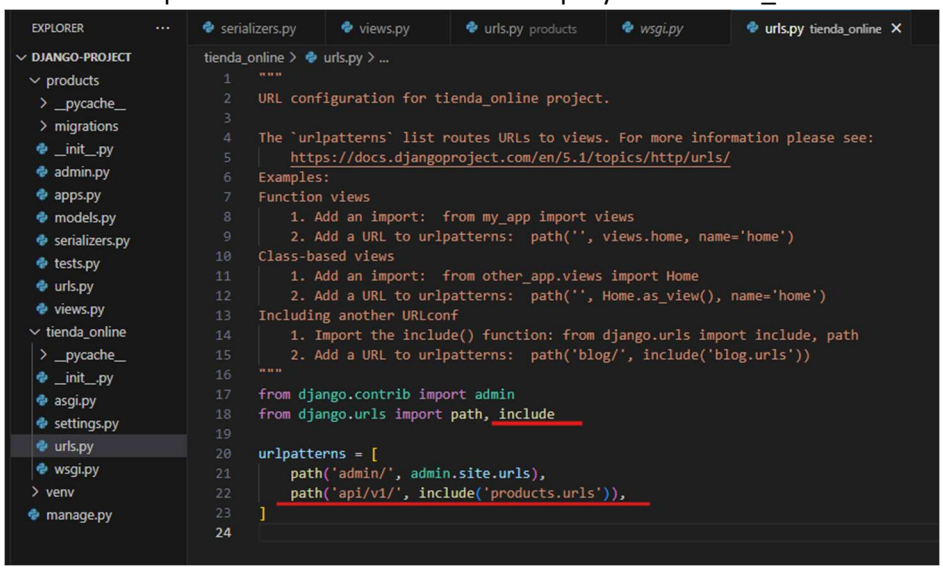
2. Modificar el archivo views.py



3. Agregar las urls
   a. Crear nuevo archivo urls.py en el folder de products



4. Incluir las url patterns en el archivo de urls del proyecto “tienda_online”



Ya está lista la API



PARTE 5: Agregar Categorias

1. Modificar el archivo models.py en el folder de products

```python
from django.db import models

# Modelo de Categorias
class Category(models.Model):
    name = models.CharField(max_length=255, verbose_name='Nombre')

    class Meta:
        verbose_name = 'Category'
        verbose_name_plural = 'Categories'
        ordering = ['name']

    def __str__(self):
        return self.name

# Modelo de productos
class Product(models.Model):
    name = models.CharField(max_length=255, verbose_name='Nombre')
    category = models.ForeignKey(Category, on_delete=models.CASCADE, related_name='get_products',
verbose_name='Categoria')
    description = models.TextField(verbose_name='Description')
    price = models.DecimalField(max_digits=10, decimal_places=2, verbose_name='Price')
    is_active = models.BooleanField(default=True)

    class Meta:
        verbose_name = 'Product'
        verbose_name_plural = 'Products'
        ordering = ['name']

    def __str__(self):
        return self.name
```

2. Modificar el archivo serializers.py en el folder de products

```python
from rest_framework import serializers
from .models import Category, Product

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ('id', 'name', 'description', 'price', 'category')
        #fields = '__all__'

class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = '__all__'
```

3. Modificar el archive de views.py en el folder de products

```python
from rest_framework import viewsets
from .models import Category, Product
from .serializers import CategorySerializer, ProductSerializer

class ProductViewSet(viewsets.ModelViewSet):
    queryset=Product.objects.all()
    serializer_class = ProductSerializer

class CategoryViewSet(viewsets.ModelViewSet):
    queryset=Category.objects.all()
    serializer_class = CategorySerializer
```

4. Modificar el archivo de urls.py en el folder de products

```python
from django.urls import path, include
from .views import CategoryViewSet, ProductViewSet
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register('products', ProductViewSet, basename = 'products' )
router.register('categories', CategoryViewSet, basename = 'categories')

urlpatterns = [
    path('', include(router.urls))
]
```

5. Crear las migraciones
   a. *py manage.py makemigrations*
   b. *py manage.py migrate*

   Si le muestra un error al momento de hacer la migración es porque estamos agregando una foreign key en el modelo de producto. Y si ya habíamos agregado algún producto a la base de datos, debemos de definir un valor por default de categoría. En caso contrario tenemos que:

   1. *Eliminar la base de datos y crearla de nuevo*
   2. *Hacer las migraciones*
   3. *Crear el super usuario*

Referencias

https://codigofacilito.com/articulos/articulo_15_10_2019_16_52_20

https://www.python.org/downloads/

https://www.django-rest-framework.org/#installation

https://www.youtube.com/watch?v=HTWlm6HkqVQ&list=PLxooeC3-xaNfS7jgZvVUM-kUcrUbgHTWJ&index=1